Chapter 4: Real-world data representation using tensors

In the previous chapter, we learned that tensors are building blocks for data in PyTorch, functioning as inputs, outputs, and parameters like weights and biases. Knowing how to perform operations on tensors and index them effectively is central to using PyTorch successfully. This chapter covers representing real-world data as PyTorch tensors, working with various data types, loading data from files, converting data to tensors, and shaping tensors to serve as inputs for neural network models.

We explore how to take raw data such as videos or text and represent them as tensors appropriate for training deep learning models. Each section covers a specific data type and dataset, starting with image data, moving to volumetric medical data, tabular wine data, time-series bike-sharing data, and text data from Jane Austen. In every section, we stop right before feeding the data into a model to prepare excellent training material for subsequent chapters.

4.1 Working with images

The introduction of convolutional neural networks revolutionized computer vision, enabling end-to-end networks trained on paired input-output examples to solve complex visual tasks. To participate in this revolution, we must load images from common formats and transform them into tensor representations arranged as PyTorch expects. An image is represented as a collection of scalars arranged in a regular grid with height and width in pixels.

4.1.1 Adding color channels

Colors are commonly encoded in RGB, where each color channel acts as a grayscale intensity map of that specific color component. Figure 4.1 illustrates a rainbow broken into its respective color channels.

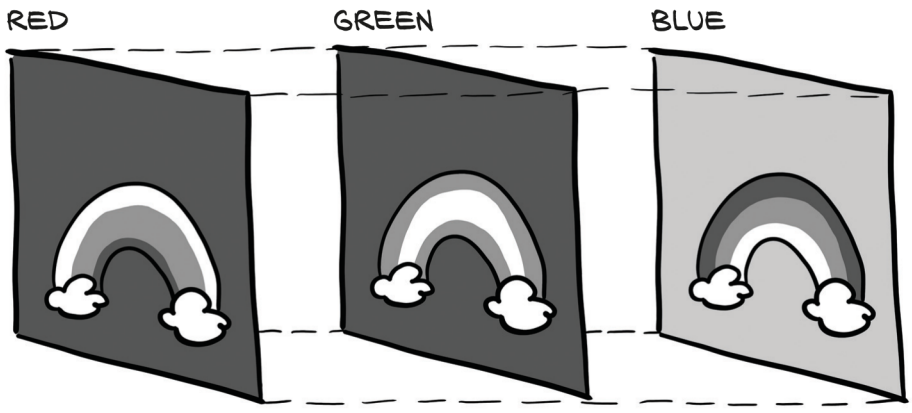


Figure 4.1 A rainbow, broken into red, green, and blue channels

4.1.2 Loading an image file

Images can be loaded in Python using various modules such as imageio.

4.1.3 Working with images

Example 1 Creating an image tensor from a NumPy array and inspecting its shape.

```
import torch
import numpy as np

torch.manual_seed(0)
rng = np.random.default_rng(0)
np_img = rng.integers(0, 256, size=(64, 64, 3), dtype=np.uint8)
img_t = torch.from_numpy(np_img)
print(img_t.shape)  # Out: torch.Size([64, 64, 3])
```

Converting NumPy arrays to PyTorch tensors bridges the gap between image loading libraries and deep learning pipelines. This zero-copy conversion preserves memory efficiency when handling large raw image arrays.

Example 2 Move the channel axis of an `H x W x C` image to the front with permute.

```
import torch

img = torch.arange(72).reshape(4, 6, 3).to(torch.uint8)
print(img.shape)  # Out: torch.Size([4, 6, 3])
chw = img.permute(2, 0, 1)
print(chw.shape)  # Out: torch.Size([3, 4, 6])
print(chw.dtype)  # Out: torch.uint8
print(img[1, 2, 0].item(), chw[0, 1, 2].item())  # Out: 24 24
```

Deep learning libraries expect channels first, while many image readers return channels last, so this reordering is one of the most common preprocessing steps. Checking shapes immediately after the call catches layout mistakes before they reach a model. Indexing the same pixel in both layouts confirms that only the labelling of the axes changed.

Example 3 Show that permute returns a view that shares storage with the original image, and only the strides change.

```
import torch

img = torch.zeros(4, 6, 3, dtype=torch.uint8)
chw = img.permute(2, 0, 1)
print(chw.data_ptr() == img.data_ptr())  # Out: True
chw[0, 0, 0] = 255
print(img[0, 0, 0].tolist())  # Out: [255, 0, 0]
print(img.stride())  # Out: (18, 3, 1)
print(chw.stride())  # Out: (1, 18, 3)
```

Because no pixels are copied, permuting even a very large image costs almost nothing. The flip side is that writing through the permuted tensor also changes the original, which can surprise code that assumes independent copies. Printing the strides is a quick way to see how the same memory

is being read in a different order.

Example 4 Preallocate an empty uint8 batch with the `N x C x H x W` shape.

```
import torch

batch_size = 3
batch = torch.zeros(batch_size, 3, 4, 6, dtype=torch.uint8)
print(batch.shape)  # Out: torch.Size([3, 3, 4, 6])
print(batch.dtype)  # Out: torch.uint8
print(batch.numel())  # Out: 216
print(batch.numel() * batch.element_size())  # Out: 216
```

Allocating the whole batch once avoids growing a container image by image. Knowing the element count and the byte size up front lets you budget memory for large batches before any data is loaded. Zeros are a safe starting value because every slot is overwritten when a real image arrives.

Example 5 Fill the preallocated batch slot by slot from a list of channels-last images.

```
import torch

images = [torch.full((4, 6, 3), 10 * (i + 1), dtype=torch.uint8)
          for i in range(3)]
batch = torch.zeros(len(images), 3, 4, 6, dtype=torch.uint8)
for i, img in enumerate(images):
    batch[i] = img.permute(2, 0, 1)
print(batch.shape)  # Out: torch.Size([3, 3, 4, 6])
print(batch[:, 0, 0, 0].tolist())  # Out: [10, 20, 30]
print(batch.sum().item())  # Out: 4320
```

Copying each permuted image straight into its slot keeps one allocation for the whole batch. The loop mirrors how data loaders assemble mini-batches from individual samples. Spot-checking one value per slot confirms the images landed in the expected order.

Example 6 Drop an alpha channel by slicing the first three channels after permuting an `H x W x 4` image.

```
import torch

rgba = torch.arange(96).reshape(4, 6, 4).to(torch.uint8)
chw = rgba.permute(2, 0, 1)
print(chw.shape)  # Out: torch.Size([4, 4, 6])
rgb = chw[:3]
print(rgb.shape)  # Out: torch.Size([3, 4, 6])
batch = torch.zeros(2, 3, 4, 6, dtype=torch.uint8)
batch[0] = rgb
print(batch[0, 2, 0, 0].item(), rgba[0, 0, 2].item())  # Out: 2 2
```

Many PNG files carry a fourth transparency channel that a three-channel model cannot accept. Slicing after the permute selects channels with a cheap view and no extra copy. Comparing one

value before and after confirms that the color channels were kept in order.

Example 7 Select only the names with a given extension, then size the batch from the selection.

```
import os
import torch

names = ["cat1.png", "notes.txt", "cat2.png", "cat3.jpg", "cat4.png"]
pngs = [n for n in names if os.path.splitext(n)[-1] == ".png"]
print(pngs)  # Out: ['cat1.png', 'cat2.png', 'cat4.png']
batch = torch.zeros(len(pngs), 3, 4, 6, dtype=torch.uint8)
for i in range(len(pngs)):
    img = torch.full((4, 6, 3), i + 1, dtype=torch.uint8)
    batch[i] = img.permute(2, 0, 1)
print(batch.shape)  # Out: torch.Size([3, 3, 4, 6])
print(batch[:, 0, 0, 0].tolist())  # Out: [1, 2, 3]
```

Real image folders often contain notes, thumbnails or other formats that must not enter the batch. Sizing the batch from the filtered list avoids both missing slots and unused slots filled with zeros. Using the file extension is a simple first filter before any file is opened.

Example 8 Handle an empty list of images by building a batch with zero slots.

```
import torch

images = []
batch = torch.zeros(len(images), 3, 4, 6, dtype=torch.uint8)
print(batch.shape)  # Out: torch.Size([0, 3, 4, 6])
print(batch.numel())  # Out: 0
print(batch.sum().item())  # Out: 0
for i, img in enumerate(images):
    batch[i] = img.permute(2, 0, 1)
print("filled", batch.shape[0], "slots")  # Out: filled 0 slots
```

Pipelines sometimes receive an empty folder or a filter that matches nothing, and a batch with a first dimension of 0 is a valid tensor. Code that handles this case explicitly can skip training or inference cleanly instead of failing later with an obscure shape error. Checking `batch.shape[0]` is a cheap guard to place before any heavy work.

Example 9 Catch a size mismatch when an image does not fit the slot of a batch.

```
import torch

batch = torch.zeros(2, 3, 4, 6, dtype=torch.uint8)
small = torch.ones(5, 7, 3, dtype=torch.uint8).permute(2, 0, 1)
try:
    batch[0] = small
except RuntimeError as e:
    print(type(e).__name__)  # Out: RuntimeError
print(batch.sum().item())  # Out: 0
```

Images in one folder do not always share the same height and width, and assigning a mismatched image into a batch raises an error. Catching the exception class lets the pipeline report the offending file and carry on. The batch stays untouched by the failed assignment, so earlier slots remain valid.

Example 10 Trigger three common failures: a permute with the wrong number of axes, a repeated axis and an out-of-range batch index.

```
import torch

img = torch.zeros(4, 6, 3, dtype=torch.uint8)
batch = torch.zeros(2, 3, 4, 6, dtype=torch.uint8)
checks = [
    ("bad permute", lambda: img.permute(1, 0)),
    ("repeated dim", lambda: img.permute(0, 0, 1)),
    ("bad index", lambda: batch[5]),
]
for label, action in checks:
    try:
        _ = action()
    except (RuntimeError, IndexError) as e:
        print(label, type(e).__name__)
        # Out: bad permute RuntimeError
        #      repeated dim RuntimeError
        #      bad index IndexError
```

Each of these mistakes produces a clear exception class, and knowing which class to expect makes error handling precise. Wrapping risky shape changes in a small loop like this makes it easy to log what failed and why. Printing only the class name keeps logs stable across library versions.

Example 11 Compare a permuted view with a contiguous copy and show that both hold identical values.

```
import torch

img = torch.arange(72).reshape(4, 6, 3).to(torch.uint8)
view = img.permute(2, 0, 1)
copy = img.permute(2, 0, 1).contiguous()
print(torch.equal(view, copy))  # Out: True
print(view.is_contiguous(), copy.is_contiguous())  # Out: False True
print(view.data_ptr() == img.data_ptr())  # Out: True
print(copy.data_ptr() == img.data_ptr())  # Out: False
batch = torch.zeros(1, 3, 4, 6, dtype=torch.uint8)
batch[0] = view
print(torch.equal(batch[0], copy))  # Out: True
```

A view avoids duplicating pixel data, which matters when images are large or batches are built many times. A contiguous copy is only needed when an operation requires a dense memory layout. Checking equality of the results shows that choosing the cheaper path does not change the data.

Example 12 Compare a loop-based batch assembly with a vectorized stack and confirm identical results.

```
import torch

arrs = [torch.full((4, 5, 3), i + 1, dtype=torch.uint8) for i in range(3)]
batch_loop = torch.zeros(3, 3, 4, 5, dtype=torch.uint8)
for i, arr in enumerate(arrs):
    batch_loop[i] = arr.permute(2, 0, 1)
stacked = torch.stack([a.permute(2, 0, 1) for a in arrs])
print(batch_loop.shape)  # Out: torch.Size([3, 3, 4, 5])
print(stacked.shape)  # Out: torch.Size([3, 3, 4, 5])
print(torch.equal(batch_loop, stacked))  # Out: True
```

`torch.stack` builds a batch in one call instead of writing into a preallocated buffer element by element, which removes a Python-level loop from the hot path. The two approaches produce identical tensors, so the vectorized version can replace the loop without changing any downstream behavior.

Example 13 Wrap the batch building in a reusable function with type hints, a docstring and input validation.

```
import torch


def images_to_batch(images: list[torch.Tensor], height: int,
                    width: int, channels: int = 3) -> torch.Tensor:
    """Stack H x W x C uint8 images into an N x C x H x W batch."""
    batch = torch.zeros(len(images), channels, height, width,
                        dtype=torch.uint8)
    for i, img in enumerate(images):
        if img.dim() != 3 or tuple(img.shape[:2]) != (height, width):
            raise ValueError(f"image {i} has shape {tuple(img.shape)}")
        if img.shape[2] < channels:
            raise ValueError(f"image {i} has too few channels")
        batch[i] = img.permute(2, 0, 1)[:channels]
    return batch


rgb = torch.full((4, 6, 3), 7, dtype=torch.uint8)
rgba = torch.full((4, 6, 4), 9, dtype=torch.uint8)
out = images_to_batch([rgb, rgba], 4, 6)
print(out.shape)  # Out: torch.Size([2, 3, 4, 6])
print(out[:, 0, 0, 0].tolist())  # Out: [7, 9]
try:
    _ = images_to_batch([torch.zeros(5, 6, 3, dtype=torch.uint8)], 4, 6)
except ValueError as e:
    print(type(e).__name__)  # Out: ValueError
```

Validating shapes before copying gives clear error messages that name the offending image. Accepting images with extra channels and keeping the first three makes the function tolerant of RGBA

files. Putting the logic in one function means every script in a project builds batches the same way.

4.1.4 Normalizing the data

Neural networks exhibit best performance when input data ranges from 0 to 1 or −1 to 1. Normalization can be achieved by dividing pixel values by 255.0 or scaling by the mean and standard deviation across each channel.

```
# In [6]:
batch = batch.float()
batch /= 255.0
```

```
# In [7]:
n_channels = batch.shape[1]
for c in range(n_channels):
    torch.mean(batch[:, c])
    mean = torch.mean(batch[:, c])
    std = torch.std(batch[:, c])
    batch[:, c] = (batch[:, c] - mean) / std
```

4.2 3D images: Volumetric data

Medical imaging applications like CT scans use sequences of images stacked along the head-to-foot axis to represent human anatomy. CT scans typically possess a single intensity channel representing tissue density, producing a 3D volume without native channel dimensions. Stacking individual slices results in a 5D tensor of shape $N \times C \times D \times H \times W$.

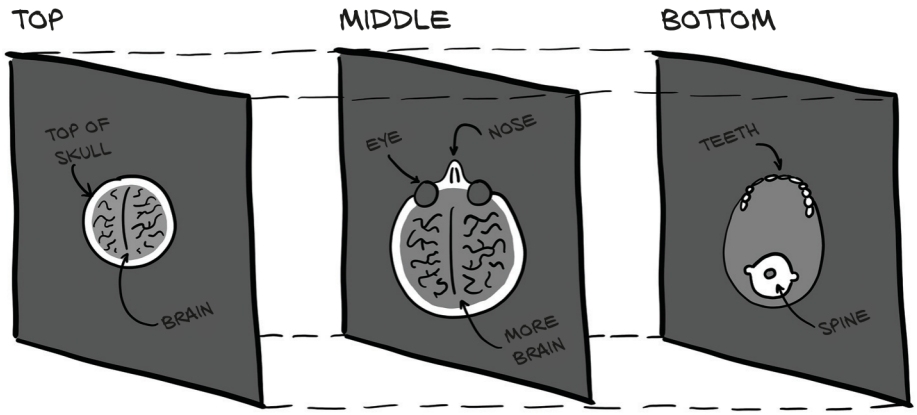


Figure 4.2 Slices of a CT scan, from the top of the head to the jawline

4.2.1 Loading a specialized format

Sample CT scans can be loaded using the volread function in imageio.

Listing 4.2 code/p1ch4/2_volumetric_ct.ipynb

```
# In [2]:
import imageio
dir_path = "../data/p1ch4/volumetric-dicom/2-LUNG 3.0 B70f-04083"
vol_arr = imageio.volread(dir_path, 'DICOM')
vol_arr.shape
# Out [2]:
Reading DICOM (examining files): 1/99 files (1.0%99/99 files (100.0%)
Found 1 correct series.
Reading DICOM (loading data): 31/99 (31.392/99 (92.999/99 (100.0%)
(99, 512, 512)
```

```
# In [3]:
vol = torch.from_numpy(vol_arr).float()
vol = torch.unsqueeze(vol, 0)
vol.shape
# Out [3]:
torch.Size([1, 99, 512, 512])
```

4.3 Representing tabular data

Tabular data consists of rows representing independent samples and columns containing specific attributes. Because PyTorch tensors are homogeneous and require numerical encoding, real-world heterogeneous tables must be converted into floating-point tensors.

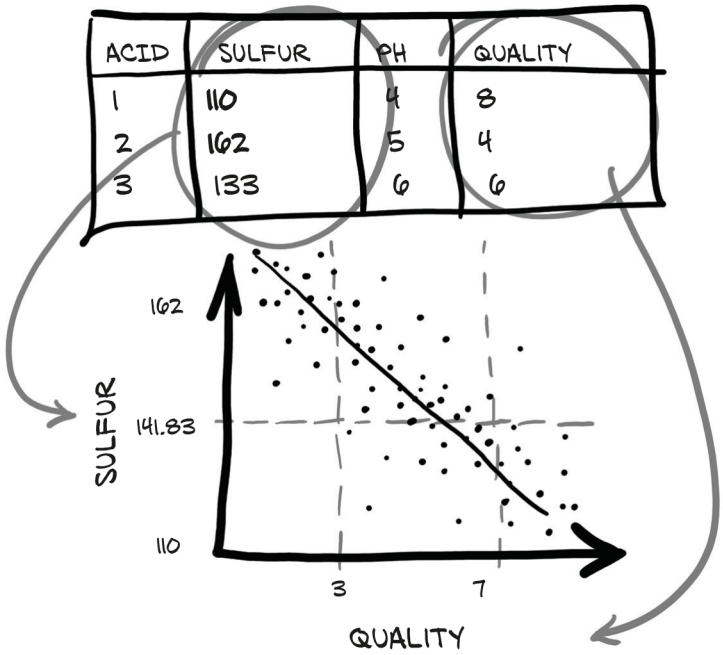


Figure 4.3 The (we hope) relationship between sulfur and quality in wine

4.3.1 Using a real-world dataset

The Wine Quality dataset contains chemical properties of vinho verde wine samples alongside a sensory quality score. The objective is to predict wine quality based on chemical attributes.

4.3.2 Loading a wine data tensor

Data can be loaded from comma-separated files using NumPy and converted into PyTorch tensors. Values are categorized into continuous, ordinal, and categorical scales.

Listing 4.3 code/p1ch4/3_tabular_wine.ipynb

```
# In [2]:
import csv
wine_path = "../data/p1ch4/tabular-wine/winequality-white.csv"
wineq_numpy = np.loadtxt(wine_path, dtype=np.float32, delimiter=";", skiprows=1)
wineq_numpy
# Out [2]:
array([[7.  , 0.27, 0.36, ..., 0.45, 8.8 , 6. ],
       [6.3, 0.3 , 0.34, ..., 0.49, 9.5 , 6. ],
       [8.1, 0.28, 0.4 , ..., 0.44, 10.1, 6. ],
       ...,
       [6.5, 0.24, 0.19, ..., 0.46, 9.4 , 6. ],
       [5.5, 0.29, 0.3 , ..., 0.38, 12.8, 7. ],
       [6.  , 0.21, 0.38, ..., 0.32, 11.8, 6. ]], dtype=float32)
```

```
# In [3]:
col_list = next(csv.reader(open(wine_path), delimiter=';'))
wineq_numpy.shape, col_list
# Out [3]:
((4898, 12),
 ['fixed acidity',
  'volatile acidity',
  'citric acid',
  'residual sugar',
  'chlorides',
  'free sulfur dioxide',
  'total sulfur dioxide',
  'density',
  'pH',
  'sulphates',
  'alcohol',
  'quality'])
```

```
# In [4]:
wineq = torch.from_numpy(wineq_numpy)
wineq.shape, wineq.dtype
# Out [4]:
(torch.Size([4898, 12]), torch.float32)
```

4.3.3 Representing scores

Quality scores are separated from input features and retained in a dedicated target tensor for regression or classification tasks.

```
# In [5]:
data = wineq[:, :-1]
data, data.shape
# Out [5]:
(tensor([[ 7.00,  0.27,  0.36, ...,  0.45,  8.80],
         [ 6.30,  0.30,  0.34, ...,  0.49,  9.50],
         [ 5.50,  0.29,  0.38, ...,  0.38, 12.80],
         ...,
         [ 6.00,  0.21,  0.38, ...,  0.32, 11.80]]), torch.Size([4898, 11]))
```

```
# In [6]:
target = wineq[:, -1]
target, target.shape
# Out [6]:
(tensor([6., 6., 7., ..., 6., 7., 6.]), torch.Size([4898]))
```

```
# In [7]:
target = wineq[:, -1].long()
target
# Out [7]:
tensor([6, 6, 7, ..., 6, 7, 6])
```

4.3.4 One-hot encoding

Categorical labels can be converted into one-hot encoded vectors via the scatter_ method. This approach eliminates artificial ordering constraints present in integer labels.

```
# In [8]:
target_onehot = torch.zeros(target.shape[0], 10)
target_onehot.scatter_(1, target.unsqueeze(1), 1.0)
# Out [8]:
tensor([[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]])
```

```
# In [9]:
target_unsqueezed = target.unsqueeze(1)
target_unsqueezed
# Out [9]:
tensor([[6],
```

```
        [6],
        [7],
        ...,
        [6],
        [7],
        [6]])
```

4.3.5 When to categorize

Continuous, ordinal, and categorical columns are handled via distinct processing strategies summarized in figure 4.4. Column variables can be normalized by subtracting their mean and dividing by the standard deviation.

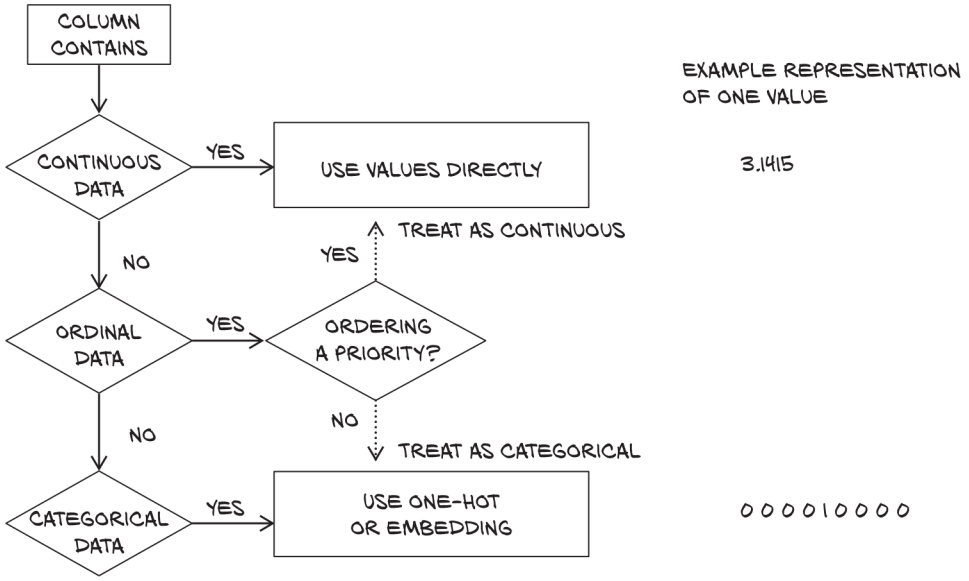


Figure 4.4 How to treat columns with continuous, ordinal, and categorical data

```
# In [10]:
data_mean = torch.mean(data, dim=0)
data_mean
# Out [10]:
tensor([6.85e+00, 2.78e-01, 3.34e-01, 6.39e+00, 4.58e-02, 3.53e+01,
        1.38e+02, 9.94e-01, 3.19e+00, 4.90e-01, 1.05e+01])
```

```
# In [11]:
data_var = torch.var(data, dim=0)
data_var
# Out [11]:
tensor([7.12e-01, 1.02e-02, 1.46e-02, 2.57e+01, 4.77e-04, 2.89e+02,
        1.81e+03, 8.95e-06, 2.28e-02, 1.30e-02, 1.51e+00])
```

```
# In [12]:
data_normalized = (data - data_mean) / torch.sqrt(data_var)
```

```
data_normalized
# Out [12]:
tensor([[ 1.72e-01, -8.18e-02,  2.16e-01, ..., -3.49e-01, -1.39e+00],
        [-6.57e-01,  2.08e-01,  5.23e-02, ...,  1.35e-03, -8.24e-01],
        [-1.61e+00,  1.17e-01,  3.43e-01, ..., -9.63e-01,  1.49e+00],
        ...,
        [-1.01e+00, -3.39e-01, -1.25e+00, ..., -6.77e-01,  1.86e+00],
        [-1.28e+00,  1.17e-01, -2.18e-01, ..., -3.49e-01,  1.04e+00],
        [-1.01e+00, -7.95e-01,  3.43e-01, ..., -9.63e-01,  3.75e-01]])
```

4.3.6 Finding thresholds

Advanced indexing with boolean tensors allows filtering dataset records to evaluate specific criteria, such as isolating bad or good wine samples based on chemical thresholds.

```
# In [13]:
bad_indexes = target <= 3
bad_indexes.shape, bad_indexes.dtype, bad_indexes.sum()
# Out [13]:
(torch.Size([4898]), torch.bool, tensor(20))
```

```
# In [14]:
bad_data = data[bad_indexes]
bad_data.shape
# Out [14]:
torch.Size([20, 11])
```

```
# In [15]:
bad_data = data[target <= 3]
mid_data = data[(target > 3) & (target < 7)]
good_data = data[target >= 7]
bad_mean = torch.mean(bad_data, dim=0)
mid_mean = torch.mean(mid_data, dim=0)
good_mean = torch.mean(good_data, dim=0)
for i, args in enumerate(zip(col_list, bad_mean, mid_mean, good_mean)):
    print('{:2} {:20} {:6.2f} {:6.2f} {:6.2f}'.format(i, *args))
# Out [15]:
0 fixed acidity         7.60   6.89   6.73
1 volatile acidity      0.33   0.28   0.27
2 citric acid           0.34   0.34   0.33
3 residual sugar        6.39   6.71   5.26
4 chlorides             0.05   0.05   0.04
5 free sulfur dioxide  53.33  35.42  34.55
6 total sulfur dioxide 170.60 141.83 125.25
7 density               0.99   0.99   0.99
8 pH                    3.19   3.18   3.22
9 sulphates             0.47   0.49   0.50
10 alcohol             10.34  10.26  11.42
```

```
# In [16]:
total_sulfur_threshold = 141.83
total_sulfur_data = data[:, 6]
predicted_indexes = torch.lt(total_sulfur_data, total_sulfur_threshold)
predicted_indexes.shape, predicted_indexes.dtype, predicted_indexes.sum()
# Out [16]:
(torch.Size([4898]), torch.bool, tensor(2727))
```

```
# In [17]:
actual_indexes = target > 5
actual_indexes.shape, actual_indexes.dtype, actual_indexes.sum()
# Out [17]:
(torch.Size([4898]), torch.bool, tensor(3258))
```

```
# In [18]:
n_matches = torch.sum(actual_indexes & predicted_indexes).item()
n_predicted = torch.sum(predicted_indexes).item()
n_actual = torch.sum(actual_indexes).item()
n_matches, n_matches / n_predicted, n_matches / n_actual
# Out [18]:
(2018, 0.74000733406674, 0.6193984039287906)
```

4.4 Working with time series

Time-series datasets include an explicit ordering across rows, enabling models to capture sequential dependencies over time. Figure 4.5 demonstrates transforming a flat 1D dataset into a structured 2D multichannel format by separating dates and hours.

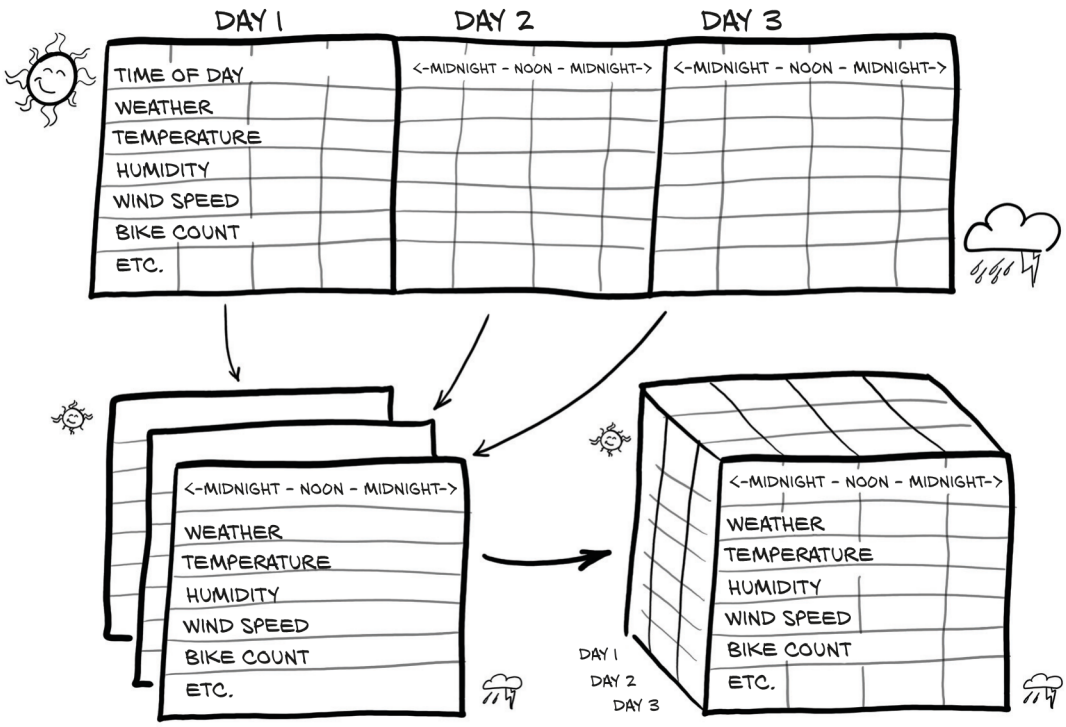


Figure 4.5 Transforming a 1D, multichannel dataset into a 2D, multichannel dataset by separating the date and hour of each sample into separate axes

4.4.1 Adding a time dimension

Sequential records are restructured into fixed-size chunks representing observation periods like days or weeks. This organizes the tensor shape into $N \times C \times L$ dimensions.

4.4.2 Shaping the data by time period

The view and transpose methods rearrange tensor shapes and strides at zero memory cost without duplicating underlying storage.

Listing 4.4 code/p1ch4/4_time_series_bikes.ipynb

```
# In [2]:
bikes_numpy = np.loadtxt(
    "../data/p1ch4/bike-sharing-dataset/hour-fixed.csv",
    dtype=np.float32,
    delimiter=",",
    skiprows=1,
    converters={1: lambda x: float(x[8:10])})
bikes = torch.from_numpy(bikes_numpy)
bikes
# Out [2]:
tensor([[1.0000e+00, 1.0000e+00, 1.0000e+00, ..., 3.0000e+00,
         1.3000e+01, 1.6000e+01],
        [2.0000e+00, 1.0000e+00, 1.0000e+00, ..., 8.0000e+00,
         3.2000e+01, 4.0000e+01],
```

```
        ...,
        [1.7378e+04, 3.1000e+01, 1.0000e+00, ..., 3.0000e+00,
         4.8000e+01, 6.1000e+01],
        [1.7379e+04, 3.1000e+01, 1.0000e+00, ..., 1.0000e+00,
         3.7000e+01, 4.9000e+01]])
```

```
# In [3]:
bikes.shape, bikes.stride()
# Out [3]:
(torch.Size([17520, 17]), (17, 1))
```

```
# In [4]:
daily_bikes = bikes.view(-1, 24, bikes.shape[1])
daily_bikes.shape, daily_bikes.stride()
# Out [4]:
(torch.Size([730, 24, 17]), (408, 17, 1))
```

```
# In [5]:
daily_bikes = daily_bikes.transpose(1, 2)
daily_bikes.shape, daily_bikes.stride()
# Out [5]:
(torch.Size([730, 17, 24]), (408, 1, 17))
```

4.4.3 Ready for training

Ordinal and categorical features like weather situations can be transformed into one-hot encoded vectors or rescaled to continuous intervals to optimize neural network training.

```
# In [6]:
first_day = bikes[:24].long()
weather_onehot = torch.zeros(first_day.shape[0], 4)
first_day[:, 9]
# Out [6]:
tensor([1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 2, 2,
        2])
```

```
# In [7]:
weather_onehot.scatter_(
    dim=1,
    index=first_day[:, 9].unsqueeze(1) - 1,
    value=1.0)
# Out [7]:
tensor([[0., 1., 0., 0.],
        [0., 1., 0., 0.],
        [0., 1., 0., 0.],
        ...,
        [0., 0., 1., 0.],
        [0., 1., 0., 0.],
```

```
        [0., 1., 0., 0.]])
```

```
# In [8]:
torch.cat((bikes[:24], weather_onehot), 1)[:1]
# Out [8]:
tensor([[1.0000e+00, 1.0000e+00, 1.0000e+00, ..., 0.0000e+00,
         0.0000e+00, 0.0000e+00]])
```

```
# In [9]:
daily_weather_onehot = torch.zeros(daily_bikes.shape[0], 4,
                                   daily_bikes.shape[2])
daily_weather_onehot.shape
# Out [9]:
torch.Size([730, 4, 24])
```

```
# In [10]:
daily_weather_onehot.scatter_(
    1, daily_bikes[:, 9, :].long().unsqueeze(1) - 1, 1.0)
daily_weather_onehot.shape
# Out [10]:
torch.Size([730, 4, 24])
```

```
# In [11]:
daily_bikes = torch.cat((daily_bikes, daily_weather_onehot), dim=1)
```

```
# In [12]:
daily_bikes[:, 9, :] = (daily_bikes[:, 9, :] - 1.0) / 3.0
```

```
# In [13]:
temp = daily_bikes[:, 10, :]
temp_min = torch.min(temp)
temp_max = torch.max(temp)
daily_bikes[:, 10, :] = ((daily_bikes[:, 10, :] - temp_min)
                         / (temp_max - temp_min))
```

```
# In [14]:
temp = daily_bikes[:, 10, :]
daily_bikes[:, 10, :] = ((daily_bikes[:, 10, :] - torch.mean(temp))
                         / torch.std(temp))
```

4.5 Representing text

Text data can be processed at the character level or word level to convert language into numerical tensor representations suitable for natural language processing models.

Listing 4.5 code/p1ch4/5_text_jane_austen.ipynb

```
# In [2]:
with open('../data/p1ch4/jane-austen/1342-0.txt', encoding='utf8') as f:
    text = f.read()
```

4.5.1 Converting text to numbers

Natural language datasets like Jane Austen's Pride and Prejudice from Project Gutenberg provide raw text inputs for tokenization and numerical encoding.

4.5.2 One-hot-encoding characters

Text is parsed into individual lines and characters are mapped to ASCII indices to create one-hot encoded character matrices.

```
# In [3]:
lines = text.split('\n')
line = lines[200]
line
# Out [3]:
"Impossible, Mr. Bennet, impossible, when I am not acquainted with him'"
```

```
# In [4]:
letter_t = torch.zeros(len(line), 128)
letter_t.shape
# Out [4]:
torch.Size([70, 128])
```

```
# In [5]:
for i, letter in enumerate(line.lower().strip()):
    letter_index = ord(letter) if ord(letter) < 128 else 0
    letter_t[i][letter_index] = 1
```

4.5.3 One-hot encoding whole words

Word-level encoding establishes a vocabulary mapping words to unique dictionary indices, producing wide one-hot encoded tensors across sentences. Figure 4.6 illustrates character-level, word-level, and embedding representations.

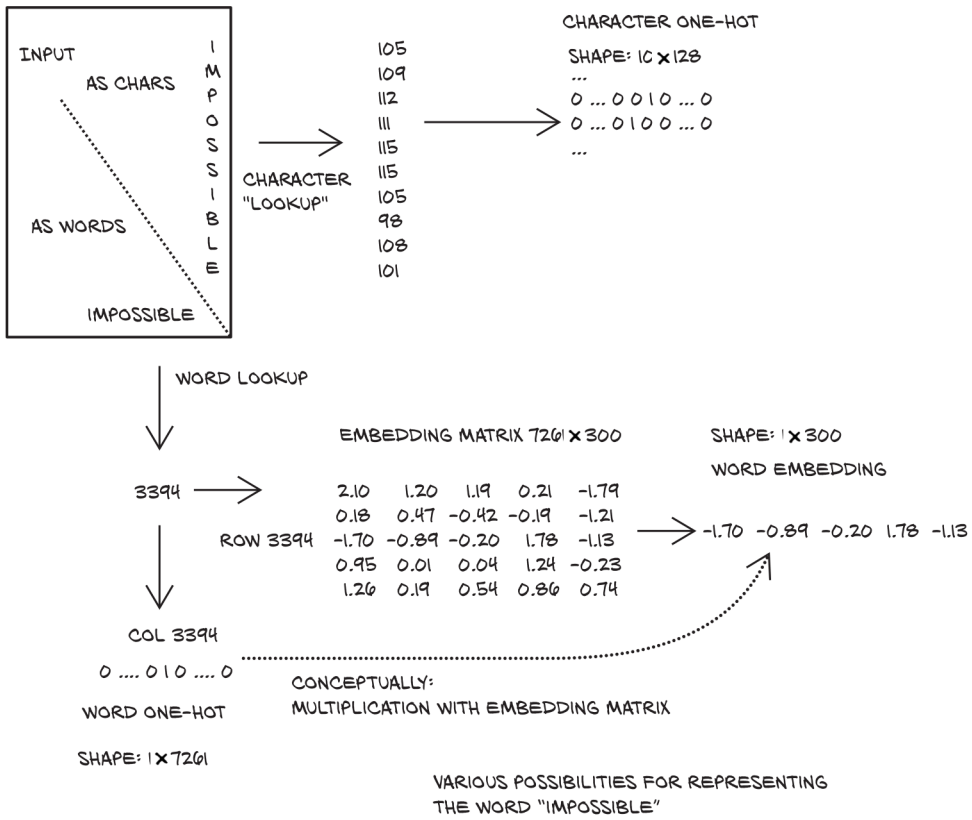


Figure 4.6 Three ways to encode a word

```
# In [6]:
def clean_words(input_str):
    punctuation = '.,;:"?!-_'
    word_list = input_str.lower().replace('\n', '').split()
    word_list = [word.strip(punctuation) for word in word_list]
    return word_list
words_in_line = clean_words(line)
line, words_in_line
# Out [6]:
("Impossible, Mr. Bennet, impossible, when I am not acquainted with him'",
 ['impossible',
  'mr',
  'bennet',
  'impossible',
  'when',
  'i',
  'am',
  'not',
  'acquainted',
  'with',
  'him'])
```

```
# In [7]:
word_list = sorted(set(clean_words(text)))
```

```
word2index_dict = {word: i for (i, word) in enumerate(word_list)}
len(word2index_dict), word2index_dict['impossible']
# Out [7]:
(7261, 3394)
```

```
# In [8]:
word_t = torch.zeros(len(words_in_line), len(word2index_dict))
for i, word in enumerate(words_in_line):
    word_index = word2index_dict[word]
    word_t[i][word_index] = 1
    print('{:2} {:4} {}'.format(i, word_index, word))
print(word_t.shape)
# Out [8]:
0 3394 impossible
1 4305 mr
2  813 bennet
3 3394 impossible
4 7078 when
5 3315 i
6  415 am
7 4436 not
8  239 acquainted
9 7148 with
10 3215 him
torch.Size([11, 7261])
```

4.5.4 Text embeddings

Embeddings map words into dense, lower-dimensional floating-point vector spaces where semantically similar words occupy neighboring regions, facilitating efficient representations and relational analogies. Figure 4.7 presents a manual 2D word embedding example.

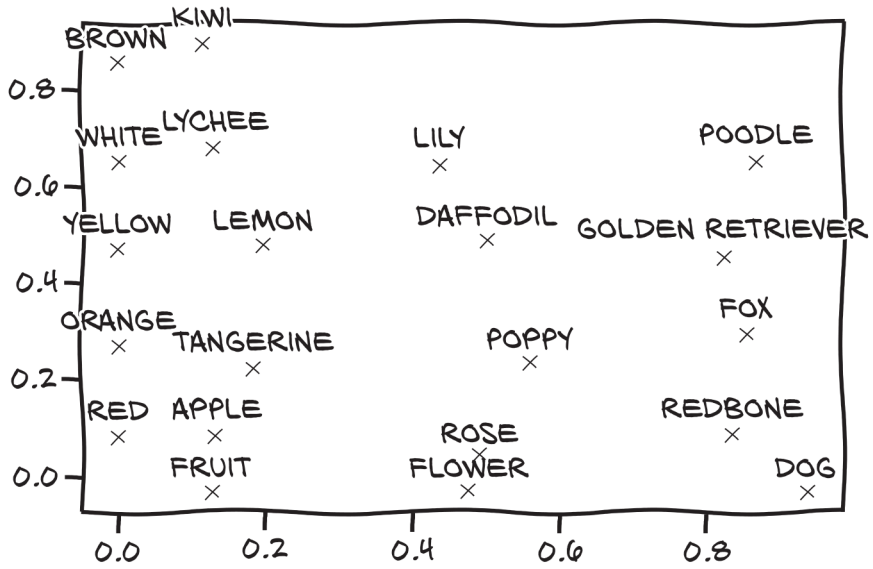


Figure 4.7 Our manual word embeddings

4.5.5 Text embeddings as a blueprint

Embeddings serve as an efficient alternative to high-dimensional one-hot encodings, applicable to text vocabularies as well as general categorical data and recommendation systems.

4.6 Conclusion

We have explored loading and shaping various real-world data types into tensor structures for neural network ingestion, laying the groundwork for training deep learning models in subsequent chapters.

4.7 Exercises

Problem 4.1 Take several pictures of red, blue, and green items with your phone or other digital camera (or download some from the internet, if a camera isn't available). a Load each image, and convert it to a tensor. b For each image tensor, use the mean() method to get a sense of how bright the image is. c Take the mean of each channel of your images. Can you identify the red, green, and blue items from only the channel averages?

Problem 4.2 Select a relatively large file containing Python source code. a Build an index of all the words in the source file (feel free to make your tokenization as simple or as complex as you like; we suggest starting with replacing r"[^a-zA-Z0-9_]+" with spaces). b Compare your index with the one we made for Pride and Prejudice. Which is larger? c Create the one-hot encoding for the source code file. d What information is lost with this encoding? How does that information compare to what's lost in the Pride and Prejudice encoding?

4.8 Summary

Neural networks require data to be represented as multidimensional numerical tensors, often 32-bit floating-point. In general, PyTorch expects data to be laid out along specific dimensions according to the model architecture—for example, convolutional versus recurrent. We can reshape data effectively with the PyTorch tensor API. Thanks to how the PyTorch libraries interact with the Python standard library and surrounding ecosystem, loading the most common types of data and converting them to PyTorch tensors is convenient. Images can have one or many channels. The most common are the red-green-blue channels of typical digital photos. Many images have a per-channel bit depth of 8, though 12 and 16 bits per channel are not uncommon. These bit depths can all be stored in a 32-bit floating-point number without loss of precision. Single-channel data formats sometimes omit an explicit channel dimension. Volumetric data is similar to 2D image data, with the exception of adding a third dimension (depth). Converting spreadsheets to tensors can be very straightforward. Categorical- and ordinal-valued columns should be handled differently from interval-valued columns. Text or categorical data can be encoded to a one-hot representation through the use of dictionaries. Very often, embeddings give good, efficient representations.